Python II 0

Εισαγωγή

Μεταβλητές & Σταθερές

- Όλες οι Γλώσσες Προγραμματισμού χρησιμοποιούν Μεταβλητές για την αποθήκευση τιμών στην μνήμη του PC.
- Μια Μεταβλητή πρέπει να έχει:
  1. Όνομα: Μια λέξη που χρησιμοποιείτε για να αναφέρεται στην τιμή που περιέχει η Μεταβλητή.
  2. Τύπος Δεδομένων: Το οποίο καθορίζει το είδος των δεδομένων που μπορεί να αποθηκεύσει η Μεταβλητή.

Τύποι Δεδομένων

Τι είναι οι Τύποι Δεδομένων;

- Σε μια γλώσσα προγραμματισμού, ένας Τύπος Δεδομένων μιας Μεταβλητής καθορίζει τι είδους τιμές μπορούν να περιέχουν και ποιες λειτουργίες μπορούν να εκτελεστούν στη Μεταβλητή.
- Οι ποιο συνηθισμένοι τύποι δεδομένων είναι οι Συμβολοσειρές, οι Αριθμοί, Booleans, οι Λίστες και οι Πίνακες.

Πρωτόγονοι Τύποι Δεδομένων

Συμβολοσειρές

- Πώς θα διακρίνουμε μεταξύ της τιμής μιας Συμβολοσειράς και του πραγματικού κώδικα του προγράμματος;
- Η λύση είναι να σηματοδοτήσετε την αρχή και το τέλος μιας συμβολοσειράς με ειδικούς χαρακτήρες.

Όπως:

τα εισαγωγικά (' ') ή τα διπλά εισαγωγικά (" ").

Πρωτόγονοι Τύποι Δεδομένων

Συμβολοσειρές:

```
print("Hello World!")
```

- Η χρήση των εισαγωγικών διασφαλίζει ότι το κείμενο ("Hello World!") αναγνωρίζεται ως Συμβολοσειρά και όχι ως δύο ξεχωριστές λέξεις που μπορεί να έχουν κάποια ιδιαίτερη σημασία στη γλώσσα προγραμματισμού.

Πρωτόγονοι Τύποι Δεδομένων

Συμβολοσειρές

- Μπορούμε να ορίσουμε μια Συμβολοσειρά με μονά εισαγωγικά ή με διπλά, αλλά όχι ανάμικτα.
- Με ότι ξεκινάμε, θα πρέπει πάλι να κλείνουμε.

Παράδειγμα:

"221051445" ή 'Καλημέρα'

Τριπλά Εισαγωγικά

- Για να μπορέσουμε να δημιουργήσουμε Συμβολοσειρές Πολλαπλών Γραμμών χρησιμοποιούμε τα Τριπλά Εισαγωγικά δηλαδή ή Τρία Μονά ή Τρία Διπλά Εισαγωγικά (""" ή ''').
- Τα Τριπλά Εισαγωγικά αναγνωρίζουν τον χαρακτήρα αλλαγής γραμμής και τον ενσωματώνουν στο κείμενο.
- Μπορούμε να χρησιμοποιήσουμε Μονά Εισαγωγικά και Διπλά Εισαγωγικά ελεύθερα μέσα στα Τριπλά Εισαγωγικά.

Τριπλά Εισαγωγικά

Για παράδειγμα:

```
X = '''
    IDLE is an integrated development
environment for Python,
which has been bundled with the default
implementation of the
language since 1.5.2b1.
'''
print(X)
```



Πρωτόγονοι Τύποι Δεδομένων

Αριθμητικοί Τύποι Δεδομένων

- Ο Δεύτερος πιο σημαντικός τύπος δεδομένων είναι τα Αριθμητικά Δεδομένα.
- Δύο από τα πιο βασικά Αριθμητικά Δεδομένα στον προγραμματισμό είναι:
  - Ακέραιοι Αριθμοί
  - Δεκαδικοί Αριθμοί

Πρωτόγονοι Τύποι Δεδομένων

- Για να ελέγξουμε τον τύπο δεδομένων μιας μεταβλητής χρησιμοποιούμε την εντολή type().

Παράδειγμα:

```
type( 1 )                    τύπος: 'int'
type( 3.14 )                 τύπος: 'float'
a = True,  type(a)           τύπος: 'bool'
type( 'python' )             τύπος: 'str'
```

Python

- Η Python είναι μια γλώσσα που ξεχωρίζει τα πεζά και τα κεφαλαία γράμματα.
- Αυτό σημαίνει ότι μπορείτε να γράψετε τις μεταβλητές, τις συναρτήσεις και της μεθόδους σας μόνο με συγκεκριμένο αποδεκτό τρόπο.
- Πχ οι μεταβλητές:

A_Variable

a_variable

A_VARIABLE

έχουν διαφορετική σημασία για την Python.

Python

- Ας γράψουμε ένα απλό πρόγραμμα Python σε ένα αρχείο.
- Πληκτρολογήστε τον ακόλουθο πηγαίο κώδικα σε αρχείο με το όνομα test.py:

```
print( "Hello, Python!" )
```

- Αποθηκεύστε και μεταγλωττίστε.

Python

- Αυτό παράγει το ακόλουθο αποτέλεσμα:



Python

- Έτσι, εάν θέλουμε να εκτυπώσουμε ένα μήνυμα στην οθόνη, χρησιμοποιούμε την εντολή print():

Σύνταξη

```
print ( "<The Message String>" )
```

Παράδειγμα:

```
print ( "Hello, Pyhton" )
```

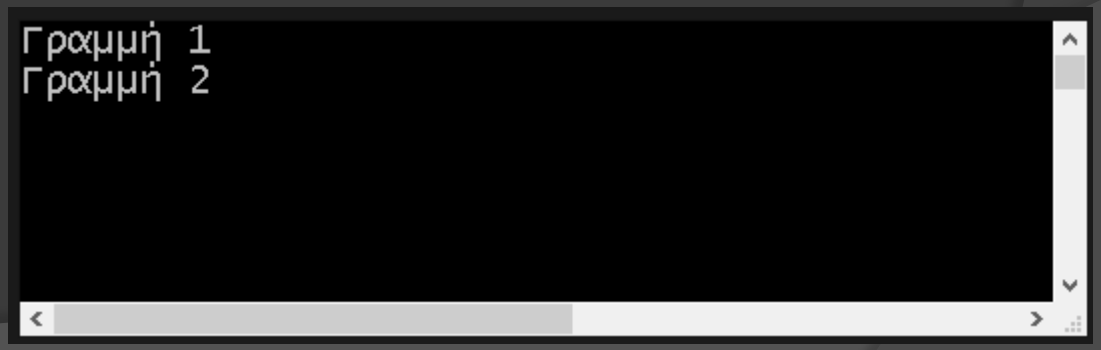
- Τα Αλφαριθμητικά (Κείμενο) πρέπει πάντα να περικλείονται μέσα σε εισαγωγικά.

Python

- Αν θέλουμε να ορίσουμε μια συμβολοσειρά δύο γραμμών ο ευκολότερος τρόπος είναι με τη χρήση του χαρακτήρα διαφυγής \n στην αρχή της νέας γραμμής.

Παράδειγμα:

```
x = "Γραμμή 1\nΓραμμή 2"
print(x)
```



Python

- Ένας ακόμα χαρακτήρας διαφυγής που είναι χρήσιμο να γνωρίζουμε είναι ο Στηλοθέτης (Tab) \t.
- Ο Στηλοθέτης αφήνει κενό ορισμένου μήκους (συνήθως τεσσάρων χαρακτήρων) μεταξύ των λέξεων μιας πρότασης με σκοπό να στοιχήσει τις λέξεις πολλαπλών γραμμών την μία κάτω από την άλλη δημιουργώντας ομοιόμορφες στήλες δεδομένων.

Python

Παράδειγμα:

```
x = "Στήλη 1\tΣτήλη 2\tΣτήλη 3\t\tΣτήλη 4"
y = "Στήλη 5\tΣτήλη 6\tΣτήλη 7\t\tΣτήλη 8"
print(x)
print(y)
```



Print()

- Στην Python δεν υπάρχει διαφορετική εντολή Εκτύπωσης όταν θέλουμε να αλλάξουμε γραμμή μετά το μήνυμα και διαφορετική αν δύο μηνύματα ή παραπάνω θέλουμε να εκτυπώνονται στην ίδια γραμμή.
- Πχ: Σαν την print() και την println() στην Java.

Print()

- Από προεπιλογή, η εντολή print() θα προσθέτει πάντα μια νέα γραμμή μετά το τέλος ενός μηνύματος.
- Αν θέλουμε να εκτυπώσουμε διαφορετικό κείμενο δίπλα-δίπλα στην ίδια γραμμή, τότε πρέπει να καθοδηγήσουμε την εντολή print() για να παραλείψετε τη εισαγωγή νέας γραμμής χρησιμοποιώντας την εντολή:

end=""

Print()

- Συντακτικό:

  print ("<*The Message String*>", end="")

- Παράδειγμα:

  print ("Hello", end="")

  print (", Pyhton")

Το αποτέλεσμα θα είναι:

Hello, Pyhton

Python

- Τώρα ας εισαγάγουμε πληροφορίες από το πληκτρολόγιο.
- Πληκτρολογήστε τον ακόλουθο πηγαίο κώδικα σε ένα αρχείο .py:

```
myAge = input("Please, Enter your Age: ")
print ("Your Age Is: ")
print(myAge)
```



Python

- Έτσι, εάν θέλουμε να εισαγάγουμε αριθμητικά δεδομένα από το πληκτρολόγιο, χρησιμοποιούμε την εντολή input():

Σύνταξη

<Variable> = input("<Prompt String>")

Παράδειγμα :

Age = input("Please, Enter your Name: ")

Η μεταβλητή "Age" θα κρατήσει την είσοδο του χρήστη.

Python

- Από προεπιλογή, η input() αποθηκεύει δεδομένα ως Αλφαριθμητικά.
- Αν θέλουμε να μετατρέψουμε τα δεδομένα από πχ. ακέραιο σε μορφή κειμένου στην αντίστοιχη αριθμητική μορφή ακέραιου, τότε θα πρέπει η επιστρεφόμενη τιμή να αλλάξει τύπο δεδομένων.
- Για τα αριθμητικά δεδομένα έχουμε τις εξής περιπτώσεις:

Python

- Ακέραιοι αριθμοί
- Πραγματικοί αριθμοί

Στο συντακτικό της Python έχουμε:

1. int: Ακέραιοι αριθμοί(2, 43, 123, 9087).
2. float: Πραγματικοί αριθμοί(2.3, 3.34, 1.987)

Python

- Έτσι, για να μετατρέψουμε τη μεταβλητή Age σε ακέραιο η πραγματικό αριθμό χρησιμοποιούμε:

- Age = int(Age) # Real to Integer
- Age = float(Age) # Integer to Real

Πχ.

```
X = input("Please, insert a number: ")
Z = int(X) * 10
```

Python

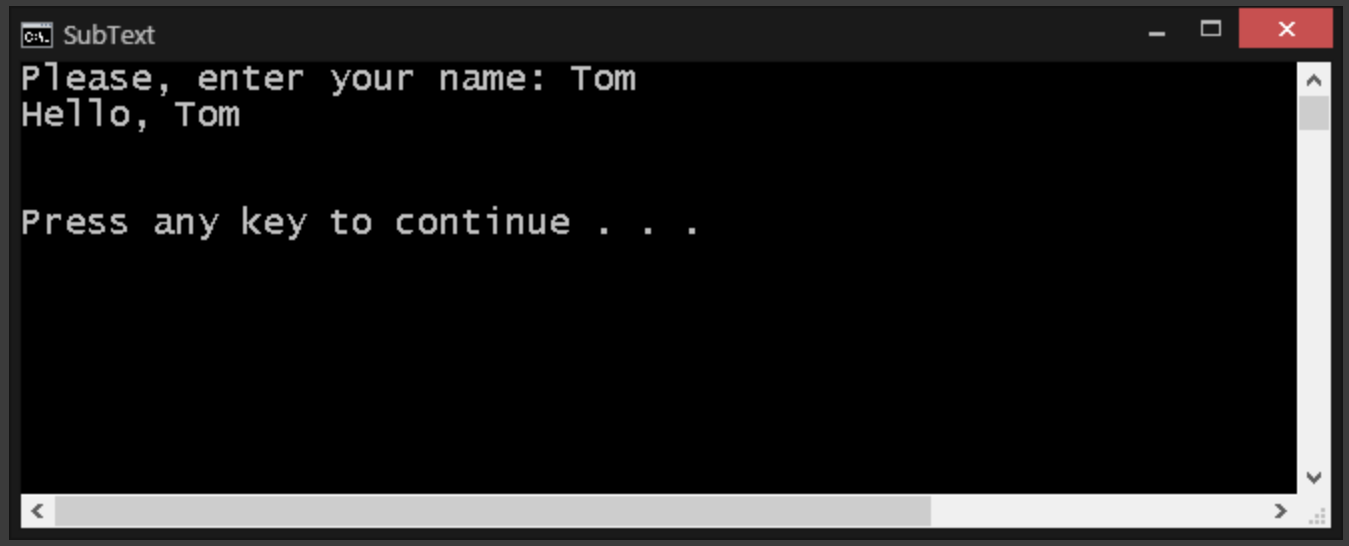
Χρησιμοποιώντας τον τελεστή συν (+) μπορούμε να συνδυάσουμε δύο ή περισσότερα αλφαριθμητικά.

Πχ.:

```
Message = "Hello, "
Name = input("Please, enter your name: ")
print(Message + Name)
```

Python

- Η έξοδος θα έχει ως εξής:



Python

- Τέλος, μπορούμε να εκτυπώσουμε πολλές μεταβλητές διαφορετικών τύπων δεδομένων χρησιμοποιώντας το διαχωριστικό κόμμα ",".

Πχ.:

```
Y = 25
Z = "Gemini"
print("I'm ", Y, " years old and I'm ", Z)
```

Python

- Η έξοδος θα έχει ως εξής:



Αριθμητικοί Τελεστές

- Ανάμεσα στην μεταβλητή A και την B μπορούμε να εισάγουμε ιδικούς χαρακτήρες που ονομάζονται Τελεστές για να πραγματοποιήσουμε διάφορων τύπων πράξεων:

Παράδειγμα:

A + B

A or B

- Η Python υποστηρίζει τους παρακάτω τύπους Τελεστών.

Αριθμητικοί Τελεστές

- Αν η μεταβλητή A = 10 και B = 20, τότε:

Operator	Description	Example
+	Προσθέτει δύο αριθμούς.	A + B = 30
-	Αφαιρεί δύο αριθμούς	A - B = -10
*	Πολλαπλασιάζει δύο αριθμούς	A * B = 200
/	Διαιρεί δύο αριθμούς	B / A = 2.0
//	Διαιρεί δύο αριθμούς με ακέραιο αποτέλεσμα.	B // A = 2
%	Το υπόλοιπο της ακέραιας διαίρεσης	B % A = 0
**	Ύψωση σε δύναμη	B ** A = 1.024

Τελεστές Ανάθεσης

- Αν η μεταβλητή A = 10 και B = 20, τότε:

Χειριστής	Περιγραφή	Παράδειγμα
=	Εκχωρεί τιμές δεξιάς από τον τελεστή στον τελεστή αριστερά του τελεστή.	c = a + b.
+=	Προσθέτει στον δεξιό τελεστή στον αριστερό τελεστή και αποδίδει το αποτέλεσμα στον αριστερό τελεστή.	c += a είναι ισοδύναμο με c = c + a
-=	Αφαιρεί το δεξί τελεστή από τον αριστερό τελεστή και αποδίδει το αποτέλεσμα στον αριστερό τελεστή.	c -= a είναι ισοδύναμο με c = c – a
*=	Πολλαπλασιάζει τον δεξιό τελεστή με τον αριστερό τελεστή και αποδίδει το αποτέλεσμα στον αριστερό τελεστή.	c *= a είναι ισοδύναμο με c = c * a
/=	Διαιρεί τον αριστερό τελεστή με το δεξί τελεστή και αποδίδει το αποτέλεσμα στον αριστερό τελεστή.	c /= a είναι ισοδύναμο με c = c / a

Σχεσιακοί Τελεστές

- Ο παρακάτω πίνακας δείχνει όλους τους σχετικούς τελεστές που υποστηρίζονται από την Python.
- Αν η μεταβλητή A = 10 και B = 20, τότε:

Χειριστής	Περιγραφή	Παράδειγμα
==	Ελέγχει εάν οι τιμές των δύο τελεστών είναι ίσες ή όχι, αν ναι, τότε η συνθήκη γίνεται αληθής.	(A == B) = false.
!=	Ελέγχει εάν οι τιμές των δύο τελεστών είναι ίσες ή όχι, εάν οι τιμές δεν είναι ίσες, τότε η συνθήκη γίνεται αληθής.	(A != B) = true.
>	Ελέγχει εάν η τιμή του αριστερά είναι μεγαλύτερη από την τιμή στα δεξιά, αν ναι, τότε η συνθήκη γίνεται αληθής.	(A > B) = false.
<	Ελέγχει εάν η τιμή του αριστερά είναι μικρότερη από την τιμή στα δεξιά, αν ναι, τότε η συνθήκη γίνεται αληθής.	(A < B) = true.
>=	Ελέγχει εάν η τιμή του αριστερά είναι μεγαλύτερη ή ίση από την τιμή στα δεξιά, αν ναι, τότε η συνθήκη γίνεται αληθής.	(A >= B) = false.
<=	Ελέγχει εάν η τιμή του αριστερά είναι μικρότερη ή ίση από την τιμή στα δεξιά, αν ναι, τότε η συνθήκη γίνεται αληθής..	(A <= B) = true

Τελεστές Boolean

- Όλοι οι παρακάτω Τελεστές δουλεύουν με Boolean τελεστές και παράγουν Boolean αποτελέσματα.
- Αν η μεταβλητή A = true και η B = false, τότε :

Χειριστής	Περιγραφή	Παράδειγμα
and	Ονομάζεται Boolean AND. Εάν και οι δύο τελεστές είναι αληθείς, τότε η συνθήκη γίνεται αληθής.	(A and B) = false.
or	Ονομάζεται Boolean OR. Εάν κάποιος από τους δύο τελεστές είναι αληθής, τότε η συνθήκη γίνεται αληθής.	(A or B) = true.
not	Ονομάζεται Boolean NOT. Χρησιμοποιείται για την αντιστροφή της λογικής κατάστασης του τελεστή του. Εάν μια συνθήκη είναι αληθής, τότε το NOT θα το κάνει ψευδές.	not (A or B) = false.

Προτεραιότητα Τελεστές

Χειριστής	Προτεραιότητα
**	Υψηλότερη
*, /,%	
+, -, or, xor	
=, <>, <, <=, >, >=, in	
not or and	Χαμηλότερη

Γραμμές και Εσοχές

Γραμμές και Εσοχές

- Η Python δεν παρέχει ειδικούς χαρακτήρες για να ορίσει ένα μπλοκ κώδικα για κλάσης και συναρτήσεις ή για ελέγχους ροής.
- Τα μπλοκ κώδικα υποδηλώνονται με εσοχή γραμμής, η οποία επιβάλλεται αυστηρά.
- Ο αριθμός των κενών στην εσοχή είναι μεταβλητός, αλλά όλες οι δηλώσεις εντός του μπλοκ πρέπει να έχουν το ίδιο μέγεθος εσοχής.

Γραμμές και Εσοχές

- Για παράδειγμα:

```
εντολή1:
    print ("True")
εντολή2:
    print ("False")
```

ΠΡΟΣΟΧΗ!

Είναι πολύ σημαντικό να θυμόμαστε ότι οι λάθος εσοχές μπορεί να προκαλέσουν σφάλματα.

Γραμμές και Εσοχές

- Άρα, το ακόλουθο μπλοκ δημιουργεί σφάλματα:

- εντολή1:
  print ("Answer")
  print ("True")
  εντολή2:
  print ("Answer")
  print ("False")

Γραμμές και Εσοχές

- Ο σωστός τρόπος για την εφαρμογή των εσοχών είναι η παρακάτω:

```
εντολή1:
    print ("Answer")
    print ("True")
εντολή2:
    print ("Answer")
    print ("False")
```

Group 1 (εντολή1)

Group 2 (εντολή2)

Γραμμές και Εσοχές

- Έτσι, στην Python όλες οι συνεχείς γραμμές με τον ίδιο αριθμό εσοχών θα σχηματίσουν ένα μπλοκ.

```
print("Hello, ")
print("Python ")
If True:
    print("version 3")   # Αυτή η γραμμή ανήκει στην if
```

Λήψη Αποφάσεων

Λήψη Αποφάσεων

- Οι Δομές Λήψης Αποφάσεων απαιτούν από τον προγραμματιστή να καθορίσει μία ή περισσότερες προϋποθέσεις που θα αξιολογηθούν ή θα δοκιμαστούν από το πρόγραμμα, μαζί με μια δήλωση ή δηλώσεις που θα εκτελεστούν εάν η συνθήκη προσδιορίζεται ότι είναι αληθής και, προαιρετικά, άλλες δηλώσεις που θα εκτελεστούν εάν η συνθήκη προσδιορίζεται ότι είναι να είναι ψευδής.

Λήψη Αποφάσεων

- Η γλώσσα προγραμματισμού Python παρέχει τους ακόλουθους τύπους δηλώσεων λήψης αποφάσεων.

Statement	Description
if	Μια δήλωση if αποτελείται από μια έκφραση Boolean που ακολουθείται από μία ή περισσότερες δηλώσεις.
if-else	Μια δήλωση if - else εκτελεί το άλλο μέρος else όταν η έκφραση Boolean είναι ψευδής.
Inline if-else	Μια ενσωματωμένη δήλωση if - else χρησιμοποιείται πιο συχνά για την εκχώρηση τιμών σε μεταβλητές.
nested if	Μπορείτε να χρησιμοποιήσετε μία if ή if - else μέσα σε μία άλλη δήλωση if - else.
elif	Μπορείτε να χρησιμοποιήσετε τις δηλώσεις if..elif για να συγκρίνετε μια μεταβλητή με πολλαπλές πιθανές τιμές.

If

Σύνταξη:

- If (συνθήκη):

  < Execute single line code >

  ή

- If (συνθήκη):

  < Execute multiline code >

  < Execute multiline code >

  < Execute multiline code >

If

Παράδειγμα:

- If (X >= 10):
  print("You Passed")

- If (X >= 10):
  print("You Passed")
  print("Well Done!")

If – Else

Σύνταξη:

- if (συνθήκη):

```
        < Execute multiline code >
        < Execute multiline code >
  else:
        < Execute multiline code >
        < Execute multiline code >
```

If – Else

- Η δήλωση else είναι μια προαιρετική δήλωση και θα μπορεί να υπάρχει μόνο μια δήλωση μετά ένα if.
- Π.χ.:

```
if (X >= 100):
    Z = X / 2
else:
    Z = X * 2
```

If – Else

Παράδειγμα:

```
amount = int( input("Enter amount: ") )
if (amount < 1000):
    discount = amount * 0.05
    print ("Discount", discount)
else:
    discount = amount * 0.10
    print ("Discount", discount)
```

Η Δήλωση Elif

- Η δήλωση elif σάς επιτρέπει να ελέγχετε μια μεταβλητή για πολλές πιθανές τιμές και να εκτελείτε ένα μπλοκ κώδικα μόλις μία από τις συνθήκες αξιολογηθεί ως αληθής.
- Παρόμοια με το else, η δήλωση elif είναι προαιρετική.
- Ωστόσο, σε αντίθεση με το else, για το οποίο μπορεί να υπάρχει το πολύ μια δήλωση, μπορεί να υπάρχει ένας αυθαίρετος αριθμός δηλώσεων elif μετά σε ένα if.

Η Δήλωση Elif

Σύνταξη:

```
if (συνθήκη 1):
        εντολές
elif (συνθήκη 2):
        εντολές
elif (συνθήκη 3):
        εντολές
else:
        εντολές
```

Η Δήλωση Elif

Παράδειγμα:

```
if (amount < 1000):
        discount = amount * 0.05
        print ("Discount", discount)
elif (amount < 5000):
        discount = amount * 0.10
        print ("Discount",discount)
else:
        discount = amount * 0.15
         print ("Discount", discount)
```

Σύστημα Εισαγωγής της Python

Εξωτερικές Βιβλιοθήκες

- Εκτός από τις Ενσωματωμένες Συναρτήσεις οι οποίες περιλαμβάνονται στη γλώσσα Python, μπορεί κανείς να βρει πληθώρα εξωτερικών Βιβλιοθηκών (Αρθρώματα ή Modules) στους διαδικτυακούς τόπους υποστήριξης της γλώσσας.
- Μια Βιβλιοθήκη είναι ένα αρχείο το οποίο περιέχει μια συλλογή από σχετικές συναρτήσεις.

Εξωτερικές Βιβλιοθήκες

- Ενδεικτικά, έχουμε την Μαθηματική βιβλιοθήκη και τις βιβλιοθήκες Γραφικών.
- Οι βιβλιοθήκες για να χρησιμοποιηθούν, θα πρέπει πρώτα να εισαχθούν στο πρόγραμμά μας με την εντολή import.

Παράδειγμα:

```
import math.
```

- Για να χρησιμοποιήσουμε Μαθηματικές Συναρτήσεις σε ένα πρόγραμμα μας.

Εξωτερικές Βιβλιοθήκες

- Συντακτικό

  import <Όνομα Βιβλιοθήκης>

Πχ.:

  import math

- Η εντολή import θα φορτώσει όλες τις διαθέσιμες συναρτήσεις που περιέχονται στη Βιβλιοθήκη.
- Εισάγουμε πάντοτε τις Βιβλιοθήκες πριν από την κύρια ενότητα του κώδικα μας, στην κορυφή του αρχείου μας.

Εξωτερικές Βιβλιοθήκες

- Για να έχουμε πρόσβαση σε μια από τις συναρτήσεις, θα πρέπει να δηλώσουμε το όνομα της Βιβλιοθήκης και το Όνομα της Συνάρτησης, χωρισμένα με μια Τελεία:

    < Βιβλιοθήκη >.< Συνάρτηση >

Πχ.:

```
import math
print( math.sqrt() )
```

Math

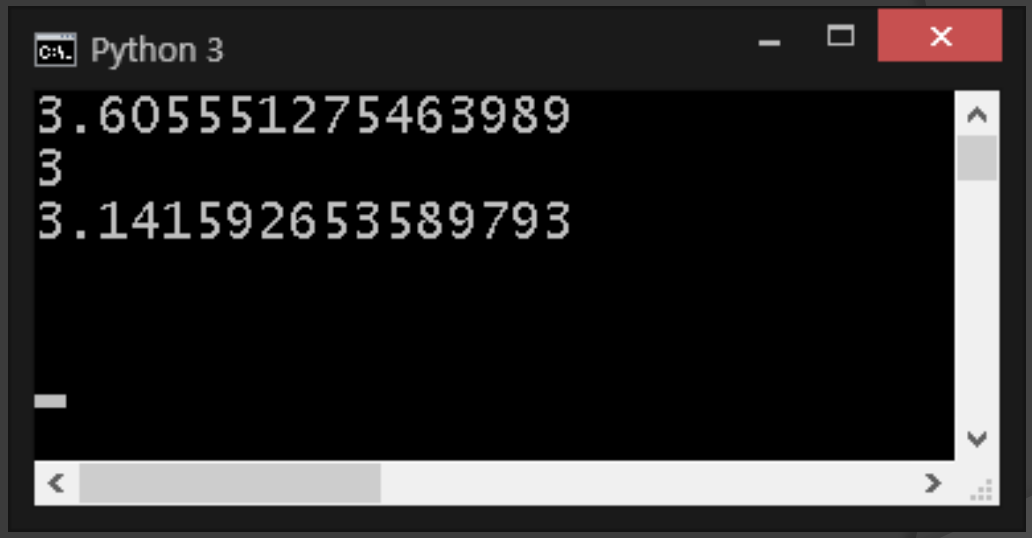
- Δύο από τις ποιο χρήσιμα εργαλεία της βιβλιοθήκης math είναι η συνάρτηση sqrt() και η σταθερά pi.
- Η sqrt() επιστρέφει την τετραγωνική ρίζα του αριθμού μέσα στις παρενθέσεις.
- Η pi επιστρέφει τον σταθερό αριθμό π (3.14...).

Math

Πχ.:

```
Import math

X = math.sqrt(13)
print( X )
print( int(X) )
Y = math.pi
print( Y )
```



Βρόχοι

- Η Python παρέχει τους ακόλουθους τύπους βρόχων.

Τύπος Βρόχου	Περιγραφή
while	Επαναλαμβάνει τις εντολές ενώ μια δεδομένη συνθήκη είναι αληθής. Ελέγχει την κατάσταση πριν εκτελέσει το σώμα του βρόχου.
for	Εκτελεί μια ακολουθία δηλώσεων πολλές φορές και συντομεύει τον κώδικα που διαχειρίζεται τη μεταβλητή βρόχου.
Nested	Μπορείτε να χρησιμοποιήσετε έναν ή περισσότερους βρόχους μέσα σε έναν οποιονδήποτε άλλο βρόχο.

Βρόχος While

- Μια δήλωση βρόχου while επιτρέπει επαναλαμβανόμενους υπολογισμούς έως ότου ικανοποιηθεί κάποια συνθήκη δοκιμής.
- Με άλλα λόγια, εκτελεί επανειλημμένα εντολές εφόσον ισχύει μια δεδομένη συνθήκη.

Σύνταξη:

```
while (condition):
    < Εντολές >
```

Παράδειγμα

```
while (count < 9):
        Sum = Sum + Number
        Number = Number - 2

print (Number)
```

range()

- Η range() είναι μια ενσωματωμένη συνάρτηση της γλώσσας Python, η οποία, ανάμεσα σε άλλα, χρησιμοποιείται για την υπόδειξη του αριθμού των επαναλήψεων που θα εκτελεστούν σε ένα βρόχο for.

Σύνταξη:

range(αρχή, μέχρι, βήμα)

Τα αρχή, μέχρι, βήμα είναι ακέραιοι αριθμοί.

range()

- Οι ενδείξεις της αρχής και του βήματος δεν είναι υποχρεωτικές και, αν δεν αναφέρονται, θα αρχίσει από τις προεπιλεγμένες τιμές δηλαδή 0 για την αρχή και θα συνεχίσει με βήμα 1.
- Αντίθετα η ένδειξη μέχρι πρέπει πάντα να αναφέρεται!

Παραδείγματα:

range(7), παράγει τη λίστα: [0,1, 2, 3, 4, 5, 6]

range(1, 7), παράγει τη λίστα: [1, 2, 3, 4, 5, 6]

range(4, -1, -1), παράγει τη λίστα [4, 3, 2, 1, 0]

range()

Σύνταξη for:

```
for Μεταβλητή_Ελέγχου in range(Τιμή):
        < Εντολές >
```

Παράδειγμα:

```
for x in range(6):
        print(" Hi :) ")
```

range()

- Σημειώστε ότι η range(6) δεν επιστρέφει τις τιμές 0 έως 6, αλλά τις τιμές 0 έως 5.

Σύνταξη:

```
for Μεταβλητή_Ελέγχου in range(αρχή, μέχρι):
        < Εντολές >
```

Παράδειγμα:

```
for x in range(2, 6):
        print(" Hi :) ")
```

Το else με Βρόχους

- Η Python υποστηρίζει την χρήση της else με μια δήλωση βρόχου.
  - Εάν η else χρησιμοποιείται με τον βρόχο for, η else θα εκτελεστεί όταν ο βρόχος έχει εξαντλήσει τις επαναλήψεις της.
  - Εάν η else χρησιμοποιείται με βρόχο while, η else εκτελείται όταν η συνθήκη γίνεται ψευδής.

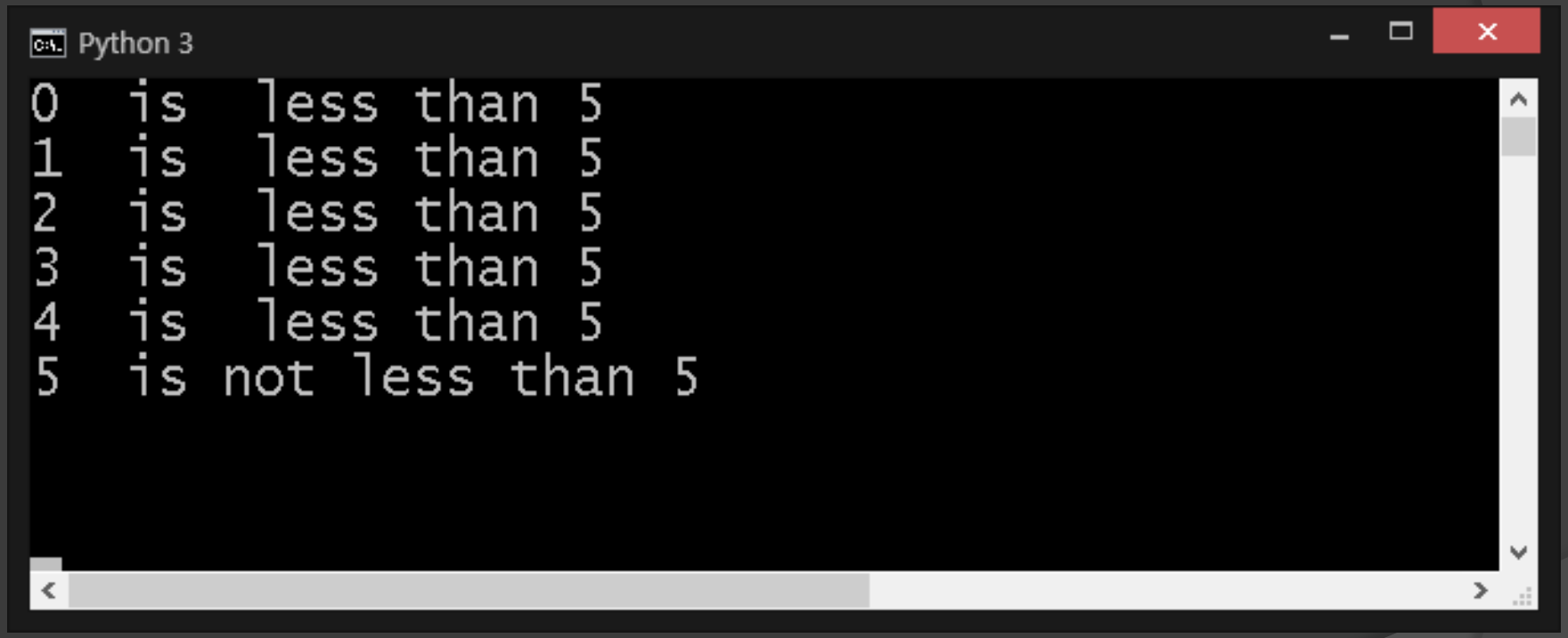
Το else με Βρόχους

- Το ακόλουθο παράδειγμα απεικονίζει τον συνδυασμό δήλωσης else με μια δήλωση βρόχου while.
- Το πρόγραμμα εκτυπώνει έναν αριθμό εφόσον είναι μικρότερος από 5, αλλιώς η δήλωση else θα εκτελεστεί εκτυπώνοντας ένα κατάλληλο μήνυμα.

Παράδειγμα

```
count = 0
while count < 5:
    print (count, " is  less than 5")
    count = count + 1
else:
    print (count, " is not less than 5")
```

Παράδειγμα


Python 3
0 is less than 5
1 is less than 5
2 is less than 5
3 is less than 5
4 is less than 5
5 is not less than 5

Ένθετοι Βρόχοι

- Στην Python και πολλές άλλες γλώσσες προγραμματισμού, οι βρόχοι είναι οι βασικές δομές για την εκτέλεση επαναλήψεων.
- Ωστόσο, μερικές φορές πρέπει να βάλετε ένα βρόχο μέσα στο άλλο, δηλαδή θα πρέπει να "φωλιάζετε" τους βρόχους.

Ένθετοι Βρόχοι

- Μπορούμε να χρησιμοποιήσουμε ένθετους for και while βρόχους σε οποιοδήποτε επίπεδο.
- Αλλά να γνωρίζετε ότι ένα βαθύ επίπεδο βρόχων καθιστά τον κώδικα δύσκολο να ελεγχθεί.
- Θα κάνει επίσης τον κώδικα δύσκολο να διατηρηθεί και να αποσφαλματωθεί.

΄Ενθετοι Βρόχοι

Παράδειγμα:

```
for x in range(6):
    for y in range(2, 10):
        for z in range(9):
            print(x, y, z)

while(x < 9):
    while(y > 30):
        print(x, y)
    print(x, y)
```

Ένθετοι Βρόχοι

- Στο επόμενο παράδειγμα, ο εξωτερικός βρόχος εκτελεί 3 επαναλήψεις (i ίση με 0, 1 και 2) και σε κάθε επανάληψη εκτελείται:

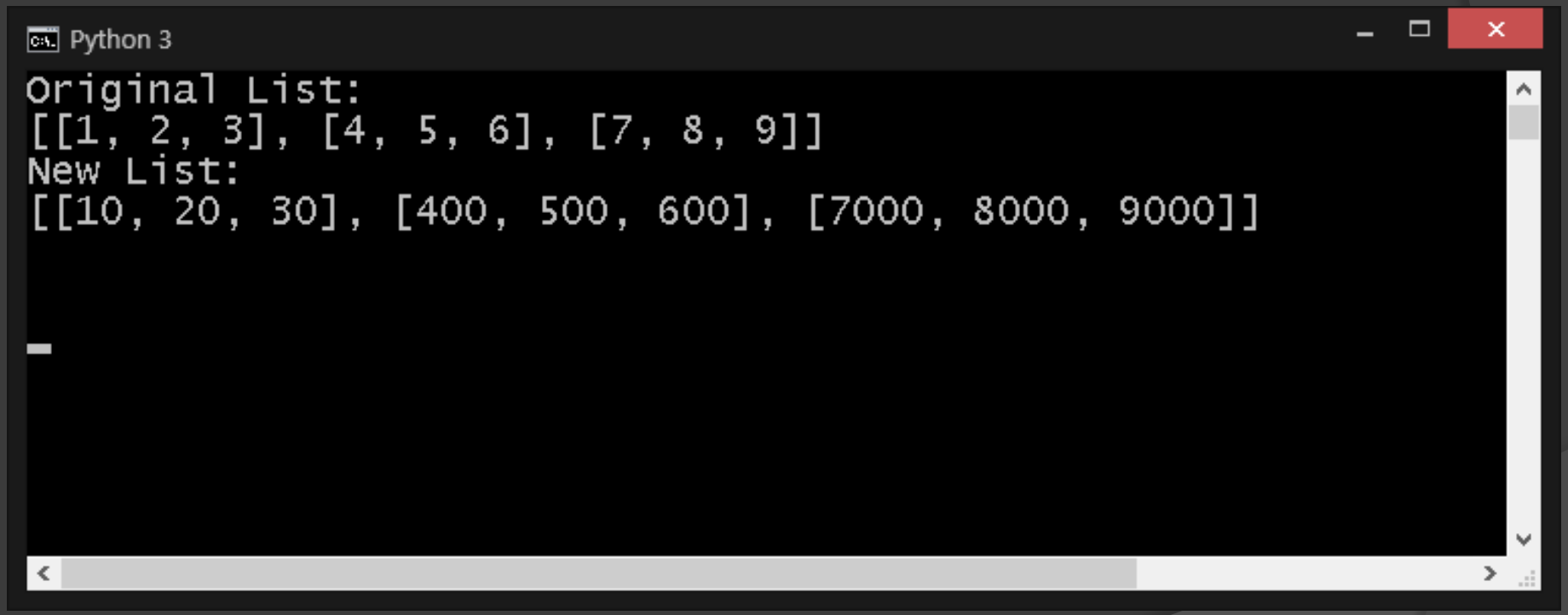
1. Ο πολλαπλασιασμός του f κατά 10.
2. Η εκτέλεση του εσωτερικού βρόχου που έχει 3 επαναλήψεις (i ίση με 0, 1 και 2) και σε κάθε επανάληψη το στοιχείο i, j του A, πολλαπλασιάζεται με το f.

Ένθετοι Βρόχοι

```
A = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
f = 1
print("Original List: ")
print(A)
for i in range(0, 3):
        f *= 10
        for j in range(0, 3):
            A[i][j] *= f
print("New List: ")
print(A)
```

Ένθετοι Βρόχοι

Το αποτέλεσμα είναι το παρακάτω:



Ένθετοι Βρόχοι

- Η σύνταξη για μια ένθετη δήλωση βρόχου for στην Python έχει ως εξής:

```
for Variable1 in List_value1:
        for Variable2 in List_value2:
                Statement(s);
```

Ένθετοι Βρόχοι

- Η σύνταξη για μια ένθετη δήλωση βρόχου στην Python έχει ως εξής:

```
while (condition1):
    while (condition2):
        Statement(s);
    Statement(s);
```

Συμβολοσειρές

- Οι συμβολοσειρές είναι από τους πιο δημοφιλείς τύπους δεδομένων στην Python.
- Μπορούμε να τα δημιουργήσουμε απλά περικλείοντας χαρακτήρες σε εισαγωγικά.
- Η Python αντιμετωπίζει τα μονά εισαγωγικά (') το ίδιο με τα διπλά εισαγωγικά (").
- πχ:

```
var1 = "Hello World!"
var2 = 'Python Programming'
```

Συμβολοσειρές

- Η Python δεν υποστηρίζει τύπο δεδομένων χαρακτήρα (Character)!
- Οι χαρακτήρες αντιμετωπίζονται ως Λίστες με μία θέση.
- Έτσι, μπορούμε να πούμε ότι μια συμβολοσειρά είναι μία Λίστα χαρακτήρων και κάθε χαρακτήρας είναι ένα υποσύνολο της συμβολοσειράς.

Συμβολοσειρές

- Η αρίθμηση των χαρακτήρων σε ένα αλφαριθμητικό ξεκινάει από το 0.

Για παράδειγμα:

word = "PYTHON"

- Η αναπαράσταση του αλφαριθμητικού μοιάζει με το παρακάτω σχήμα:

P	Y	T	H	O	N
0	1	2	3	4	5

Συμβολοσειρές

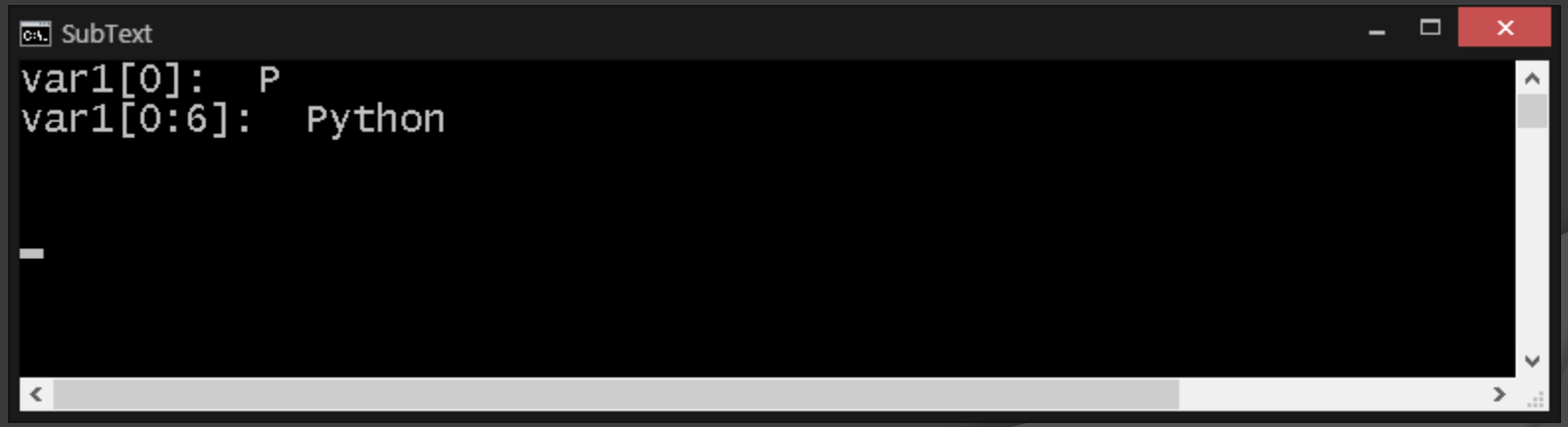
- Για παράδειγμα, το στοιχείο word[3] αναφέρεται στο 4ο γράμμα ('H'), ενώ το στοιχείο word[0], στο 1ο γράμμα ('P').
- Ο τύπος των αλφαριθμητικών στην Python ονομάζεται str, από τα αρχικά της λέξης String.

P	Y	T	H	O	N
0	1	2	3	4	5

Συμβολοσειρές

- πχ.:

```
var1 = "Python Programming"
print ("var1[0]: ", var1[0])
print ("var1[1:5]: ", var1[1:5])
```



Συμβολοσειρές

- Η Python μας προσφέρει έναν αρκετά μεγάλο αριθμό από Τελεστές και Μεθόδους οι οποίοι μας βοηθούν να διαχειριστούμε τα αλφαριθμητικά, αλλά και οποιοδήποτε άλλων τύπο δεδομένων Ακολουθίας ευκολότερα και δίχως μεγάλο κόπο.
- Αρα, για παράδειγμα εάν η μεταβλητή A είναι ίσο με "Hello" και η μεταβλητή B είναι ίσο με "Python", τότε:

Συμβολοσειρές

Τελεστής	Περιγραφή	Παράδειγμα
+	Συνδυασμός Ενώνει τις τιμές των δύο Συμβολοσειρών σε μια.	A + B δίνει: HelloPython
*	Επανάληψη Δημιουργεί νέες συμβολοσειρές, συνδυάζοντας πολλαπλά αντίγραφα της ίδιας συμβολοσειράς.	A*2 δίνει: HelloHello
[]	Διαχωρισμός Επιστρέφει τον χαρακτήρα από μια θέση	A[1] δίνει: e
[:]	Τελεστής Διαμέρισες (:) Επιστρέφει τους χαρακτήρες από μια σειρά θέσεων	A[1:4] δίνει: ell B[2:] δίνει: thon A[:2] δίνει: llo B[-3:] δίνει: hon
in	Υπαρξιακός Τελεστής Επιστρέφει true εάν υπάρχει ο χαρακτήρας στη μέσα στην συμβολοσειρά	"H" in A δίνει:True
not in	Αντίστροφος Υπαρξιακός Τελεστής Επιστρέφει true εάν ο χαρακτήρας δεν υπάρχει στην συμβολοσειρά	"M" not in A δίνει: True

Λίστες

- Η Λίστα είναι ο πιο ευέλικτος τύπος δεδομένων που είναι διαθέσιμος στην Python, ο οποίος μπορεί να γραφτεί ως Σειρά τιμών διαχωρισμένων με κόμμα (στοιχεία) μέσα σε αγκύλες.
- Σημαντικό για μια Λίστα είναι ότι τα στοιχεία σε μια Λίστα δεν χρειάζεται να είναι του ίδιου τύπου.
- Η δημιουργία μιας Λίστας είναι τόσο απλή όσο η τοποθέτηση διαφορετικών τιμών διαχωρισμένων με κόμμα μεταξύ αγκυλών.

Λίστες

- πχ.:

```
list1 = ["physics", "chemistry", 1997, 2000];
list2 = [1, 2, 3, 4, 5 ];
list3 = ["a", "b", "c", "d"];
```

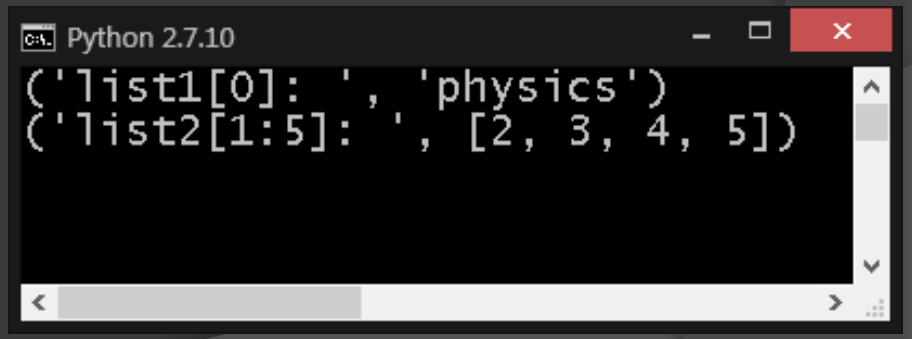
- Παρόμοια με τους δείκτες συμβολοσειρών, οι δείκτες λίστας ξεκινούν από το 0 και οι λίστες μπορούν να τεμαχιστούν, να συνδυαστούν και ούτω καθεξής.

Λίστες

- Για να αποκτήσετε πρόσβαση στις τιμές μιας λίστας, χρησιμοποιούμε τις αγκύλες [] μαζί με τον δείκτη για αυτήν την θέση.
- πχ.:

```
list1 = ["physics", "chemistry", 1997, 2000]
list2 = [1, 2, 3, 4, 5, 6, 7 ]

print ("list1[0]: ", list1[0])
print ("list2[1:5]: ", list2[1:5])
```



Λίστες

- Μπορείτε να ενημερώσετε μεμονωμένα ή πολλαπλά στοιχεία Λιστών δίνοντας το την νέα τιμή μετά τον τελεστή ίσον (=).
- πχ.:

```
list = ["physics", "chemistry", 1997, 2000]
print ("Value at index 2 : ", list[2])

list[2] = 2001
print ("New value at index 2 : ", list[2])
```

Λίστες



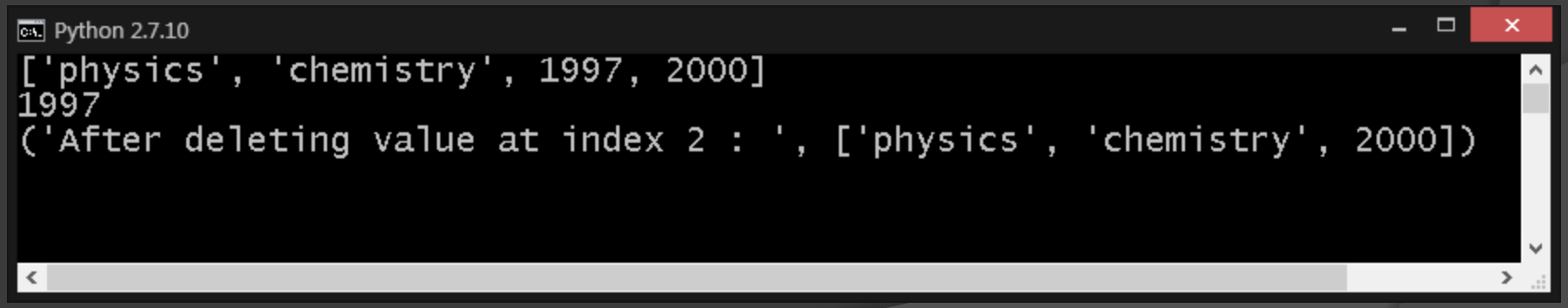
Λίστες

- Για να καταργήσετε ένα στοιχείο από την Λίστα, μπορείτε να χρησιμοποιήσετε είτε τη μέθοδο pop() εάν γνωρίζετε ακριβώς τους δείκτες των στοιχείων που θέλετε να διαγράψετε και τη μέθοδο remove() εάν δεν γνωρίζετε ακριβώς τους δείκτες από τα στοιχεία που θέλετε να διαγράψετε.
- Η μέθοδος pop() θα επιστρέψει επίσης το αντικείμενο το οποίο αφαιρει και μπορούμε να το αποθηκεύσουμε σε μεταβλητή για μεμονωμένη επεξεργασία.

Λίστες

- Πχ.:

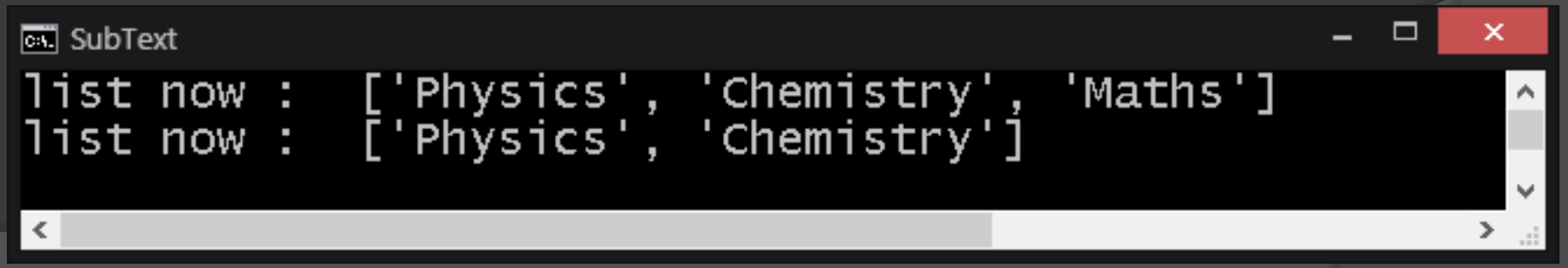
```
list = ["physics", "chemistry", 1997, 2000]
print list
print list.pop(2)
print ("After deleting value at index 2 : ", list)
```



Λίστες

- Η μέθοδος remove() δεν επιστρέφει καμία τιμή αλλά καταργεί το δεδομένο στοιχείο από τη Λίστα.
- e.g.:

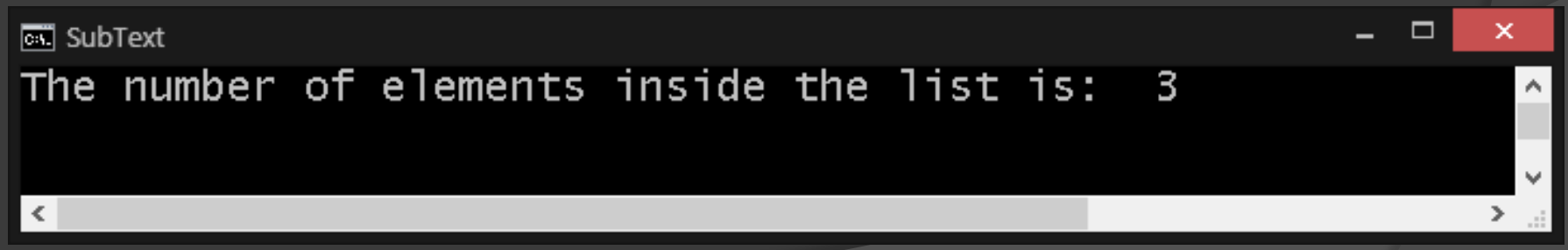
```
list1 = ["Physics", "Biology", "Chemistry", "Maths"]
list1.remove("Biology")
print ("list now : ", list1)
list1.remove("Maths")
print ("list now : ", list1)
```



Λίστες

- Η μέθοδος len() επιστρέφει τον αριθμό των στοιχείων μέσα στη Λίστα.
- Πχ.:

```
list = ["Physics", "Chemistry", "Math"]
print("The number of elements inside the list
                                    is: ",  len( list1 ) )
```



Λίστες

- Οι Λίστες ανταποκρίνονται στους τελεστές + και * σαν τις Συμβολοσειρές.
- Έχουν την ίδια σημασία (Συνένωση και Επανάληψη), με την διαφορά ότι το αποτέλεσμα θα είναι μια νέα Λίστα και όχι μια Συμβολοσειρά.

Εκφραση	Αποτελέσματα	Περιγραφή
len([1, 2, 3])	3	Μήκος
[1, 2, 3] + [4, 5, 6]	[1, 2, 3, 4, 5, 6]	Συνδυασμός
["Hi!"] * 4	["Hi!", "Hi!", "Hi!", "Hi!"]	Επανάληψη
3 in [1, 2, 3]	True	Ιδιότητα Μέλους

Πλειάδες

- Μια Πλειάδα είναι μια ακολουθία αμετάβλητων αντικειμένων της Python.
- Οι πλειάδες είναι αλληλουχίες, ακριβώς όπως και οι Λίστες.
- Η κύρια διαφορά είναι ότι οι Πλειάδες δεν μπορούν να αλλάξουν σε αντίθεση με τις Λίστες.
- Οι Πλειάδες χρησιμοποιούν παρενθέσεις, ενώ οι λίστες χρησιμοποιούν Αγκύλες.

Πλειάδες

- Η δημιουργία μιας Πλειάδας είναι τόσο απλή όσο η τοποθέτηση διαφορετικών τιμών που χωρίζονται με κόμματα μέσα σε ένα Ζευγάρι Παρενθέσεων.

- Πχ.:

```
tup1 = ("physics", "chemistry", 1997, 2000)
tup2 = (1, 2, 3, 4, 5 )
tup3 = ("a", "b", "c", "d")
```

Πλειάδες

- Για να αποκτήσουμε πρόσβαση στις τιμές της Πλειάδας, χρησιμοποιούμε τις αγκύλες μαζί με τον δείκτη ή τους δείκτες.

```
tup1 = ("physics", "chemistry", 1997, 2000)
tup2 = (1, 2, 3, 4, 5 )
print ("tup1[0]: ", tup1[0])
print ("tup2[1:5]: ", tup2[1:5])
```

Πλειάδες

- Όταν εκτελείται ο παραπάνω κώδικας, παράγει το ακόλουθο αποτέλεσμα:



Πλειάδες

- Οι Πλειάδες είναι αμετάβλητες, πράγμα που σημαίνει ότι δεν μπορείτε να ενημερώσετε ή να αλλάξετε τις τιμές των στοιχείων τους.
- Είστε όμως σε θέση να πάρετε τμήματα των υφιστάμενων Πλειάδων για να δημιουργήσετε νέες Πλειάδες.

- Πχ, Ας πούμε ότι έχουμε δύο Πλειάδες:

tup1 and tup2

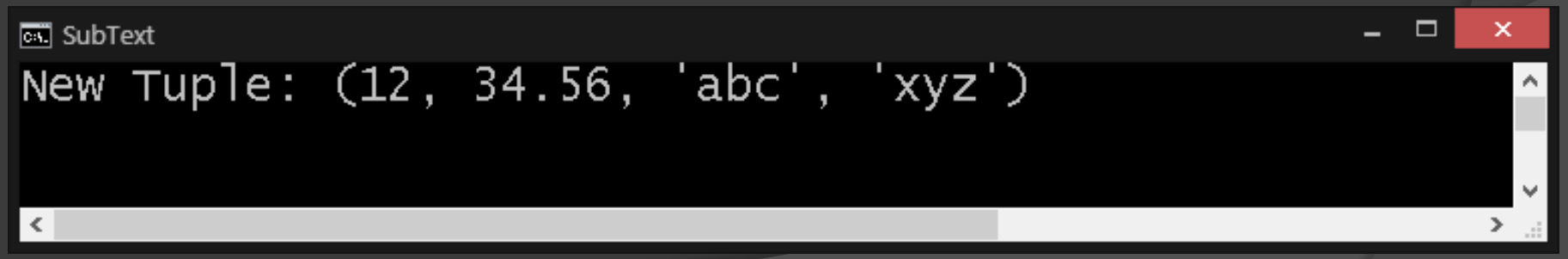
Τα παρακάτω είναι απολύτως έγκυρα…

Πλειάδες

```
tup1 = (12, 34.56)
tup2 = ("abc", "xyz")

# Ας δημιουργήσουμε μια νέα Πλειάδα ως εξής:

tup3 = tup1 + tup2
print ("New Tuple:", tup3)
```



Πλειάδες

- Η αφαίρεση μεμονωμένων στοιχείων μιας Πλειάδας δεν είναι δυνατή.
- Υπάρχει, φυσικά, η δυνατότητα να δημιουργήσουμε μια άλλη Πλειάδα από την αρχική δίχως τα ανεπιθύμητα στοιχεία.
- Για να διαγράψετε ρητά μια ολόκληρη Πλειάδα, απλά χρησιμοποιούμε την εντολή del.

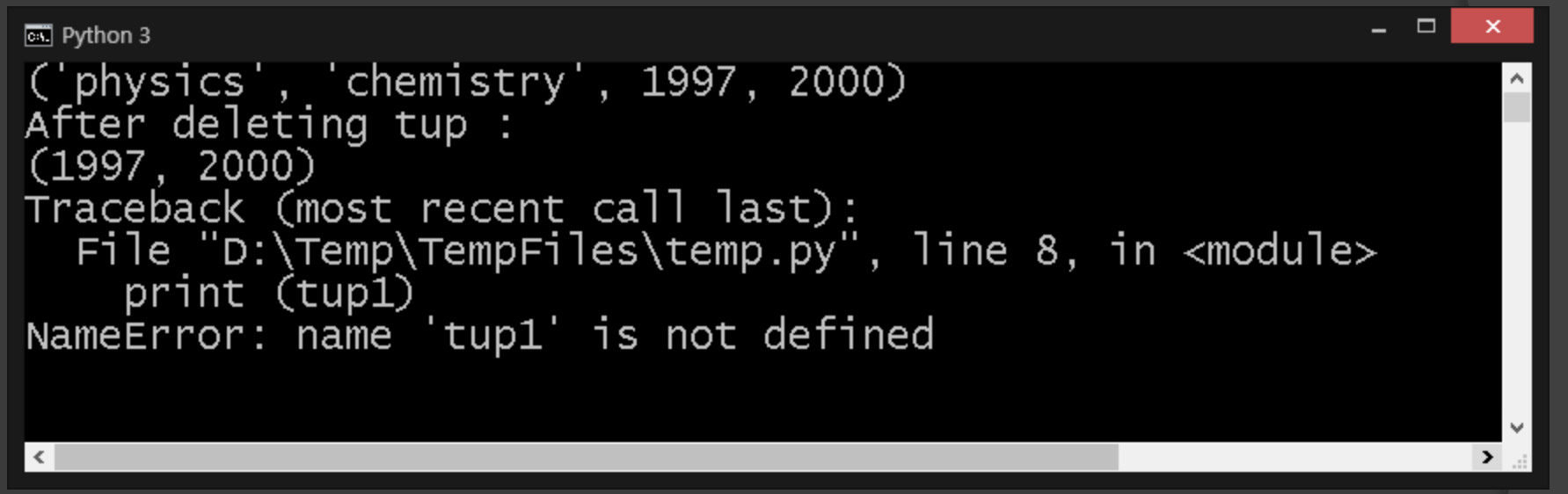
Πλειάδες

```
tup1 = ("physics", "chemistry", 1997, 2000)

print (tup1)
tup2 = tup1[2:4]
del (tup1)
print ("After deleting tup : ")
print (tup2)
print (tup1)
```

Πλειάδες

- Αυτό παράγει το ακόλουθο αποτέλεσμα.



- Το Μήνυμα Σφάλματος οφείλεται στο γεγονός ότι μετά το del(tup1), η Πλειάδα δεν υπάρχει πια!

Λεξικά

- Τα Λεξικά βελτιστοποιούν την πρόσβαση σε πληροφορίες όταν είναι γνωστό εκ των προτέρων το Κλειδί.
- Τα λεξικά της Python δημιουργούνται με την χρήση Αγκίστρων.
- Κάθε Κλειδί χωρίζεται από την Τιμή του με την χρήση της άνω-κάτω τελείας (:), τα Στοιχεία διαχωρίζονται μεταξύ τους με κόμματα.

Λεξικά

- Ένα κενό Λεξικό χωρίς κανένα στοιχείο γράφεται με την χρήση ενός άδειου ζευγαριού από Άγκιστρα.

Πχ.:

```
# Λεξικό με δύο στοιχεία
dict1 = {"Name": "Zara", "Age": 7}

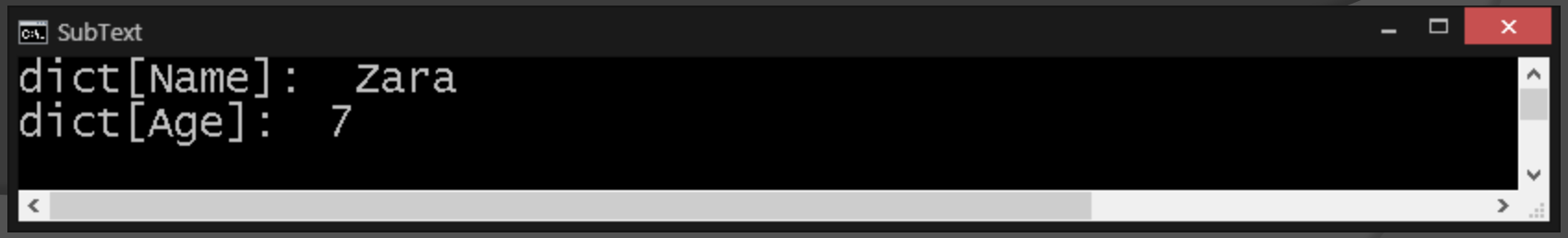
# Κενό Λεξικό
dict2 = {}
```

Λεξικά

- Για να αποκτήσουμε πρόσβαση σε ένα στοιχείο του λεξικού, μπορούμε να χρησιμοποιήσουμε τις αγκύλες μαζί με το κλειδί του στοιχείου.

Πχ.:

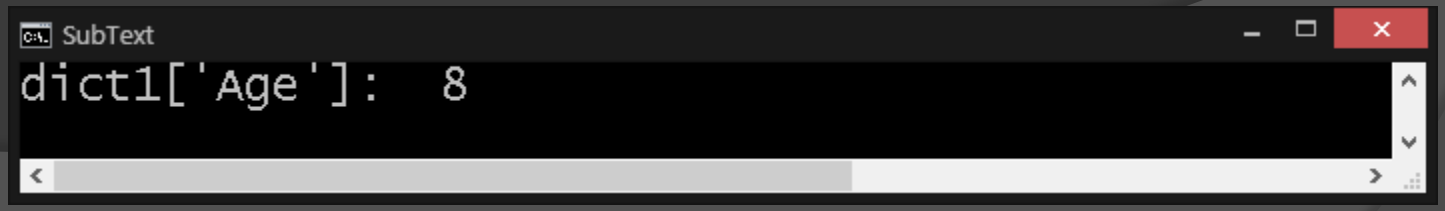
```
dict1 = {"Name": "Zara", "Age": 7}
print ("dict[Name]: ", dict1["Name"])
print ("dict[Age]: ", dict1["Age"])
```



Λεξικά

- Μπορείτε να ενημερώσετε ένα Λεξικό προσθέτοντας μια νέα καταχώρηση ή ένα νέο ζεύγος τιμών τροποποιώντας μια υπάρχουσα καταχώρηση ή την διαγραφή μιας υπάρχουσας καταχώρησης. Πχ.:

```
dict1 = {"Name": "Zara", "Age": 7}
dict1['Age'] = 8;
print ("dict1['Age']: ", dict1['Age'])
```



Λεξικά

1. Δεν επιτρέπονται περισσότερες από μία καταχωρήσεις ανά Κλειδί, άρα αυτό σημαίνει ότι Δεν Επιτρέπεται η εισαγωγή ίδιου ονόματος κλειδιού στην λίστα πολλές φορές. Όταν συναντώνται Διπλότυπα Κλειδιά, η τελευταία αντιστοίχιση υπερισχύει.

Πχ.:

```
dict1 = {"Name": "Pepa", "Age": 7, "Name": "Zara"}
print ("dict1['Name']: ", dict1['Name'])
```



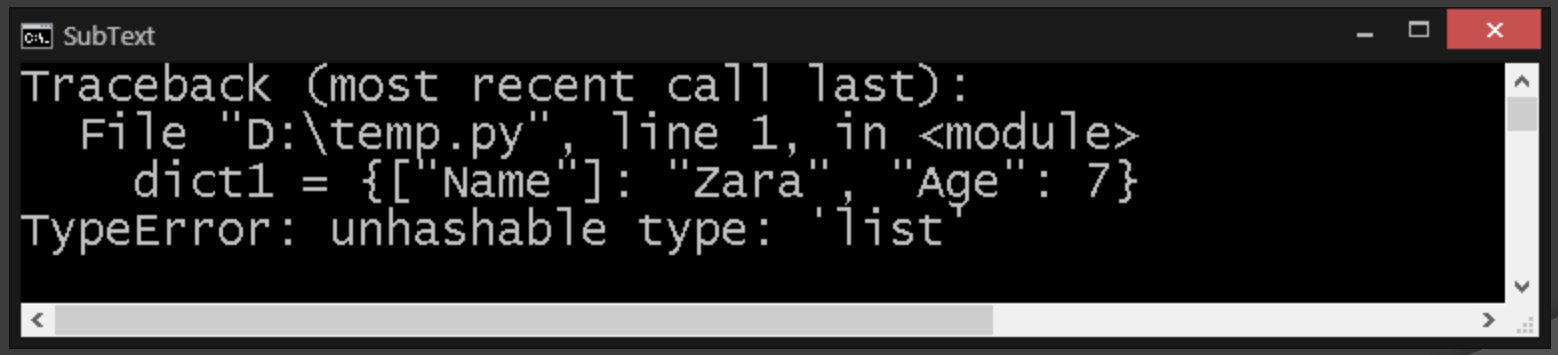
Λεξικά

2. Τα Κλειδιά πρέπει να είναι αμετάβλητα. Αυτό σημαίνει ότι μπορείτε να χρησιμοποιήσετε Κείμενο, Αριθμούς ή Πλειάδες ως Κλειδιά ενός Λεξικού, αλλά κάτι σαν πχ. μια Λίστα ["κλειδί"] δεν επιτρέπεται να εισαχτεί ως Κλειδί στην Λίστα μας.

Λεξικά

Πχ.:

```
dict1 = {["Name"]: "Zara", "Age": 7}
print ("dict['Name']: ", dict['Name'])
```



Λεξικά

Η Python περιλαμβάνει τις ακόλουθες μεθόδους για τα Λεξικά:

Μέθοδος	Περιγραφή
len(dict)	Δίνει το συνολικό μήκος του λεξικού. Αυτό θα ήταν ίσο με τον αριθμό των στοιχείων στο λεξικό.
str(dict)	Παράγει μια εκτυπώσιμη αναπαράσταση σε μορφή συμβολοσειράς ενός λεξικού.
type(variable)	Επιστρέφει τον τύπο της μεταβλητής που έχει περάσει. Εάν η μεταβλητή που έχει περάσει είναι λεξικό, τότε θα επιστρέψει έναν τύπο λεξικού.

Λεξικά

Η Python περιλαμβάνει επίσης τις ακόλουθες μεθόδους για τα Λεξικά:

Μέθοδος	Περιγραφή
dict.clear()	Διαγράφει όλα τα στοιχεία του λεξικού dict
dict.copy()	Επιστρέφει ένα αντίγραφο του λεξικού dict
dict.items()	Επιστρέφει μια λίστα ζευγαριών πλειάδων (Key, Value) από το λεξικό dict
dict.keys()	Επιστρέφει μια λίστα με τα κλειδιά του λεξικού dict
dict.update(dict2)	Προσθέτει τα στοιχεία του dict2 στο dict
dict.values()	Επιστρέφει μια λίστα με τις τιμές του λεξικού dict

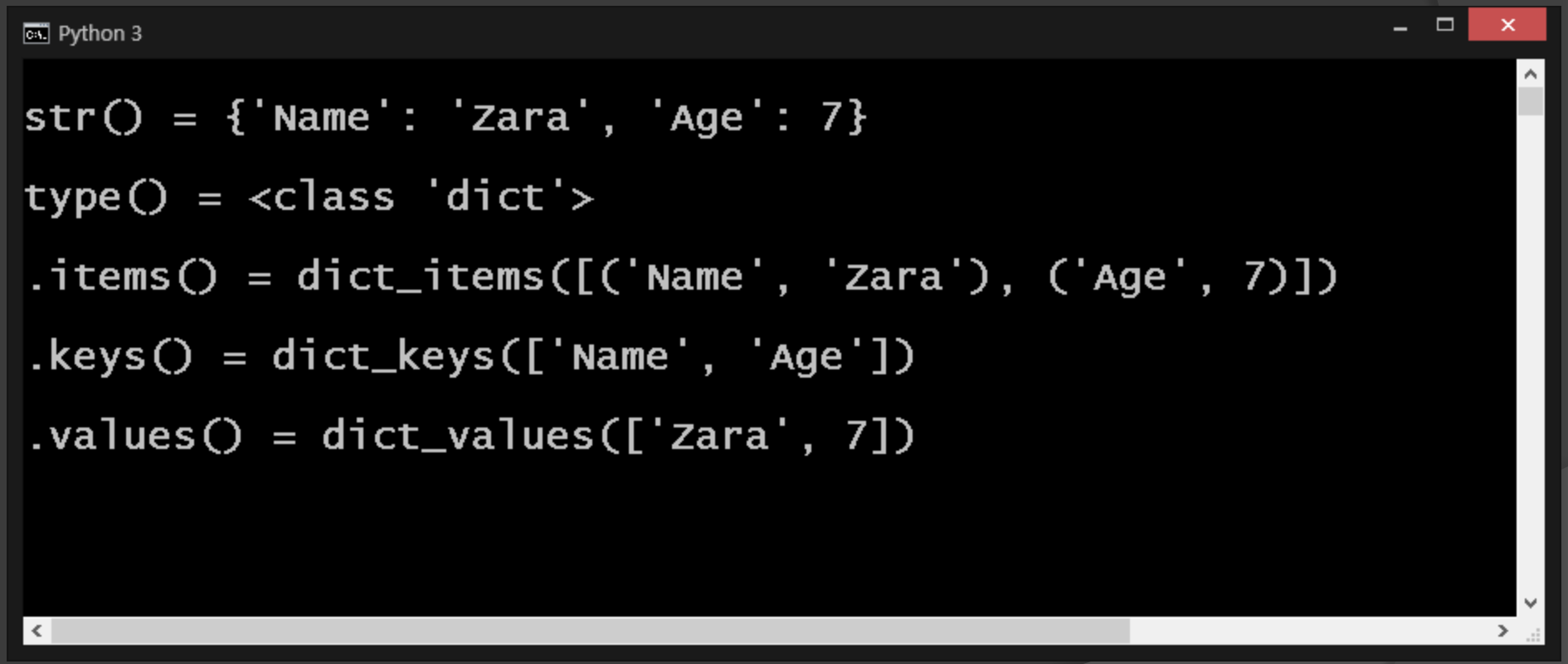
Λεξικά

Παραδείγματα:

```
dict1 = {"Name": "Zara", "Age": 7}
print("str() = "+str(dict1))
print("type() = "+str(type(dict1)))
print(".items() = "+str(dict1.items()))
print(".keys() = "+str(dict1.keys()))
print(".values() = "+str(dict1.values()))
```

Λεξικά

Και το αποτέλεσμα:



AND και OR

- Στην περίπτωση του and, η έκφραση επιστρέφει True μόνο εάν επιστρέφουν True όλες οι επιμέρους συγκρίσεις.
- Στην περίπτωση του or, η έκφραση επιστρέφει True εάν επιστρέφει True έστω και μία από τις επιμέρους συγκρίσεις.

ΠΧ.:

True and True = True, True and False = False

False or False = False, True or False = True

AND και OR

Συντακτικό:

- if ((σύγκριση1) and (σύγκριση2)) :
  < Κώδικας Python >
  else:
  < Κώδικας Python >

AND και OR

Συντακτικό:

- if ((σύγκριση1) or (σύγκριση2)) :
  < Κώδικας Python >
  else:
  < Κώδικας Python >

NOT

- Το not είναι λίγο διαφορετικό από τους τελεστές not & or και είναι γνωστός και ως "αντιστροφέας" αφού δέχεται μόνο μία Boolean είσοδο προς επεξεργασία και επιστρέφει το αντίθετο της ως έξοδο.
- Εάν, για παράδειγμα, το αποτέλεσμα μιας έκφρασης είναι True, το not θα αντιστρέψει το αποτέλεσμα σε False!
- Άρα, ο λογικός τελεστής not εξάγει πάντα το αντίστροφο της εισόδου.

NOT

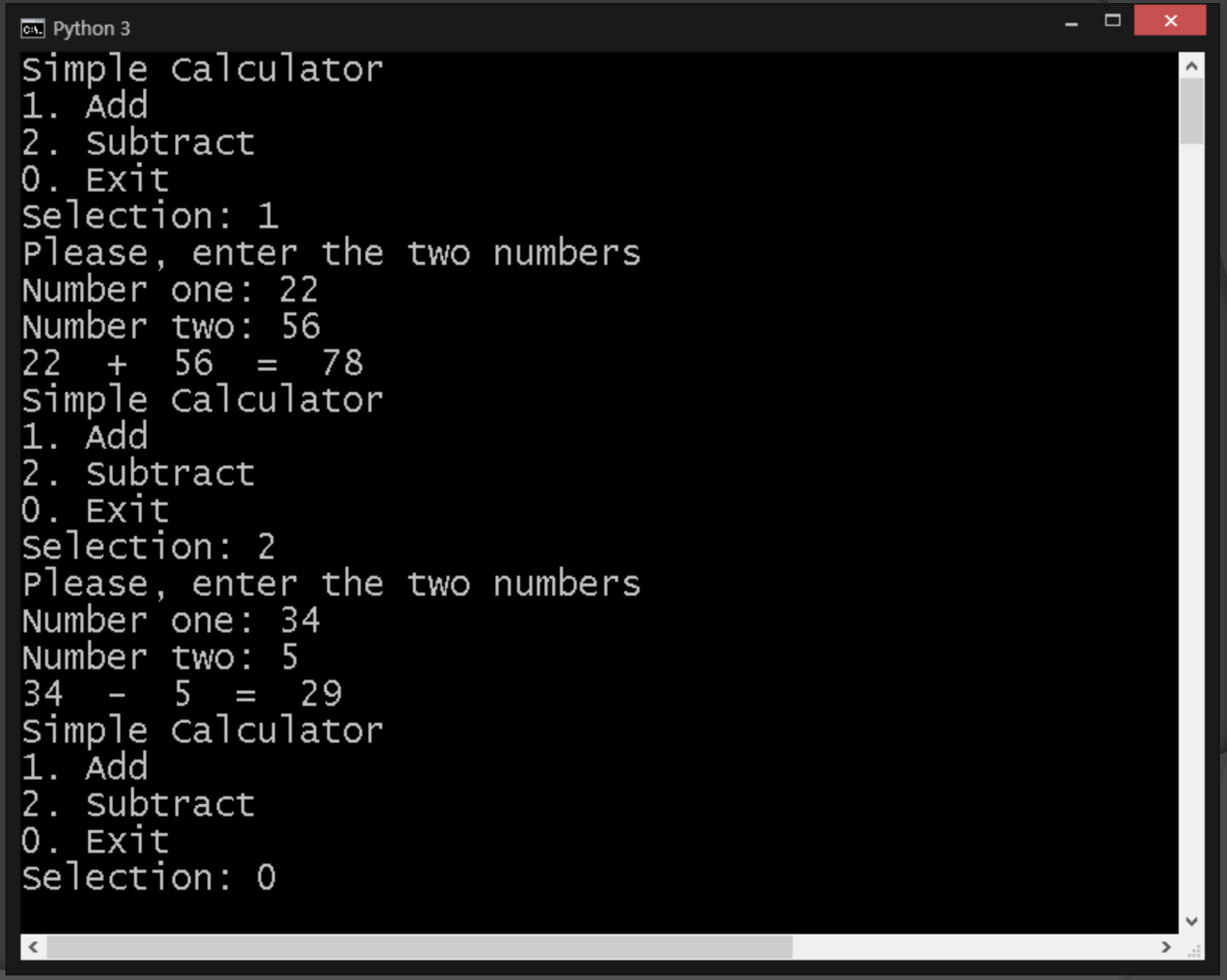
Συντακτικό:

- if (not (σύγκριση)):
  < Κώδικας Python >
  else:
  < Κώδικας Python >

Ασκήσεις

Άσκηση 1

- Δημιουργήστε μια απλή αριθμομηχανή μόνο για πρόσθεση και αφαίρεση που θα ζητά από τον χρήστη να εισάγει δύο ακέραιους αριθμούς και θα του εμφανίζει ένα απλό μενού.
- Ο χρήστης θα πρέπει να επιλέξει να προσθέσει ή να αφαιρέσει τους δύο αριθμούς.
- Το πρόγραμμα θα τερματιστεί μόνο εάν ο χρήστης διαλέξει το 0 από το μενού.


Python 3
Simple Calculator
1. Add
2. Subtract
0. Exit
Selection: 1
Please, enter the two numbers
Number one: 22
Number two: 56
22 + 56 = 78
Simple Calculator
1. Add
2. Subtract
0. Exit
Selection: 2
Please, enter the two numbers
Number one: 34
Number two: 5
34 - 5 = 29
Simple Calculator
1. Add
2. Subtract
0. Exit
Selection: 0

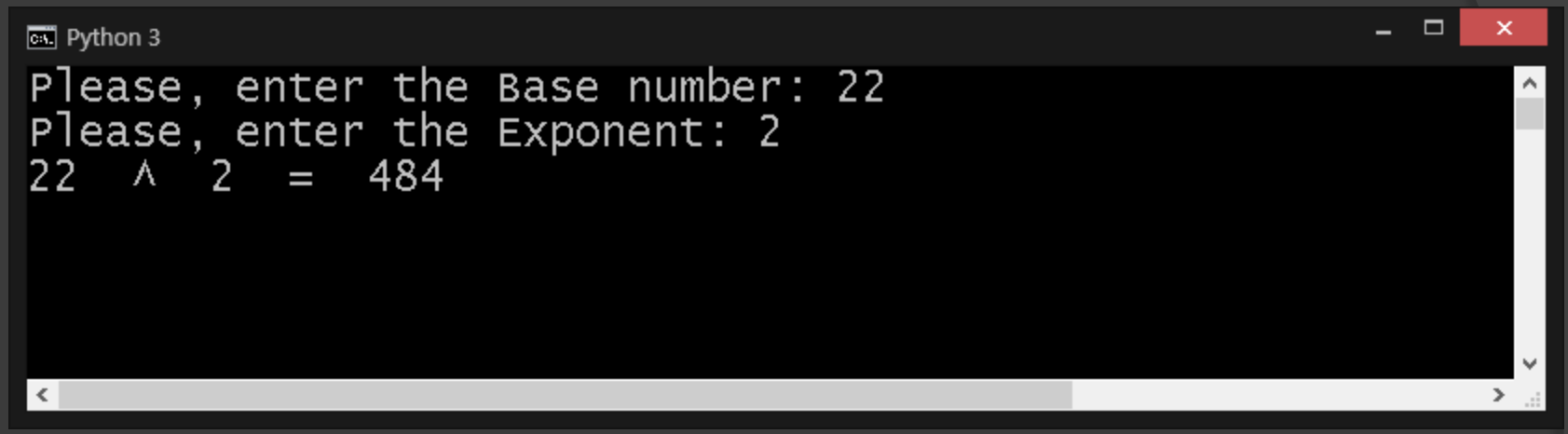
Άσκηση 2

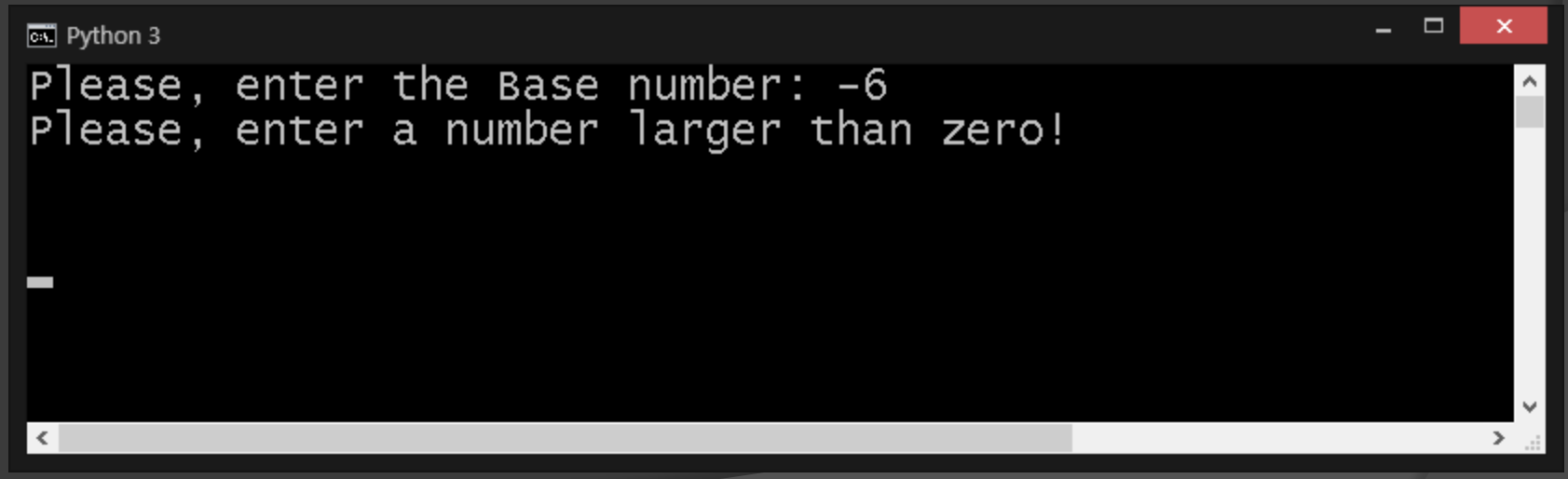
- Δημιουργήστε ένα πρόγραμμα για τον υπολογισμό της δύναμη ενός αριθμού μεγαλύτερου από το 0.
- Το πρόγραμμα θα ζητά από τον χρήστη να εισαγάγει δύο ακέραιους αριθμούς, έναν για τη βάση και έναν για τον εκθέτη.

  Παράδειγμα: βάση = 5, εκθέτης = 3 => 5^3 = 125

- Εκτυπώστε στην οθόνη το αποτέλεσμα με ένα κατάλληλο μήνυμα.

Άσκηση 2

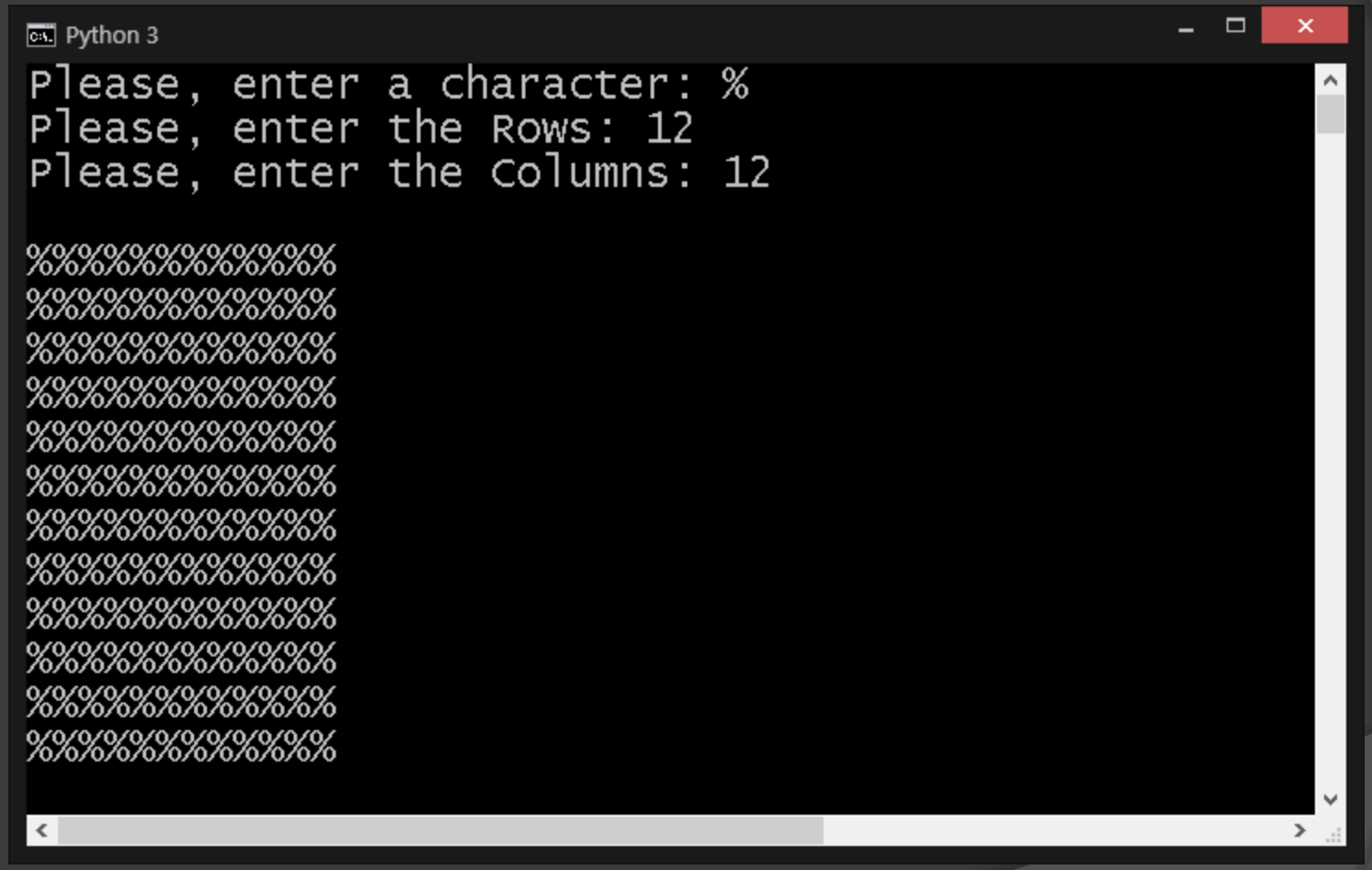

Python 3
Please, enter the Base number: 22
Please, enter the Exponent: 2
22 ^ 2 = 484


Python 3
Please, enter the Base number: -6
Please, enter a number larger than zero!

Άσκηση 3

- Δημιουργήστε ένα απλό πρόγραμμα που θα ζητά από τον χρήστη να εισάγει ένα σύμβολο από το πληκτρολόγιό του και δύο αριθμούς, COL και ROW.
- Το πρόγραμμα θα γεμίσει ένα τμήμα της οθόνης με το επιλεγμένο σύμβολο.
- Η οθόνη χωρίζεται σε Γραμμές και Στήλες.
- Το σύμβολο θα γεμίσει την περιοχή της οθόνης από τη γραμμή 1 έως τη γραμμή ROW και από τη στήλη 1 έως τη στήλη COL.

Άσκηση 3

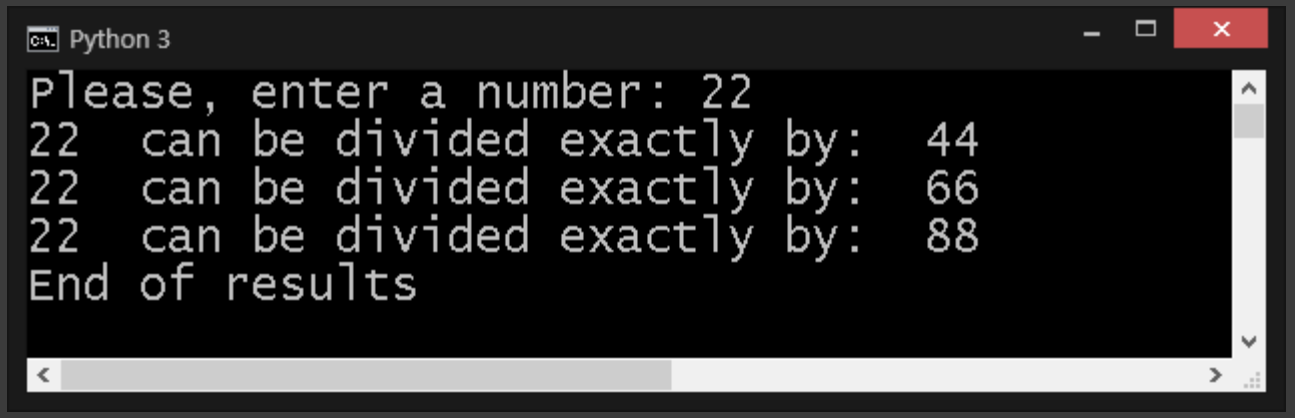

Python 3
Please, enter a character: %
Please, enter the Rows: 12
Please, enter the Columns: 12
%%%%%%%%%%%%
%%%%%%%%%%%%
%%%%%%%%%%%%
%%%%%%%%%%%%
%%%%%%%%%%%%
%%%%%%%%%%%%
%%%%%%%%%%%%
%%%%%%%%%%%%
%%%%%%%%%%%%
%%%%%%%%%%%%
%%%%%%%%%%%%
%%%%%%%%%%%%

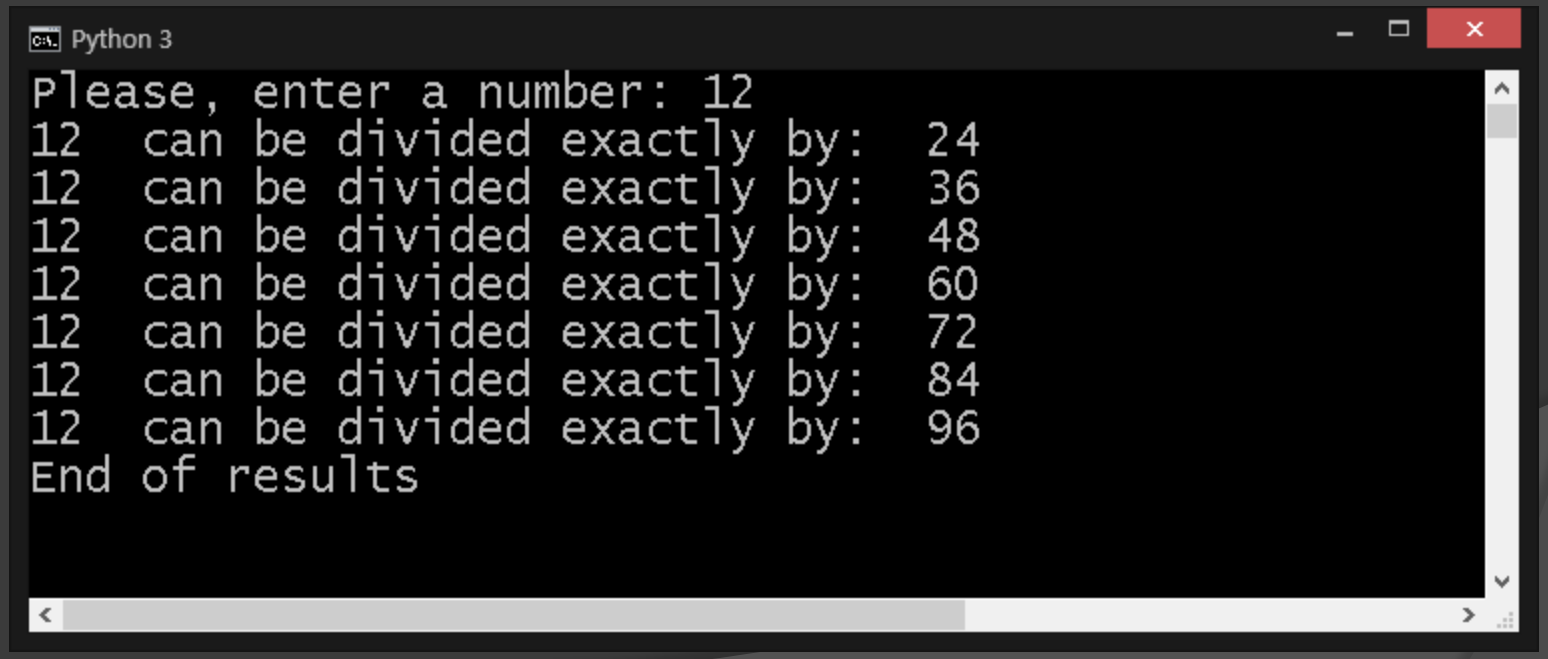
Άσκηση 4

- Δημιουργήστε ένα πρόγραμμα που θα ζητά από τον χρήστη έναν ακέραιο αριθμό μικρότερο του 100.
- Το πρόγραμμα θα εμφανίζει στην οθόνη όλους τους αριθμούς μεταξύ του 1 και του 100 που μπορούν να διαιρεθούν ακριβώς από τον αριθμό του χρήστη.

1. Ο αριθμός δεν πρέπει να είναι μικρότερος ή ίσος με 0.
2. Το πρόγραμμα δεν πρέπει να εμφανίζει στην οθόνη τον αριθμό του χρήστη.

Άσκηση 4


Python 3
Please, enter a number: 22
22 can be divided exactly by: 44
22 can be divided exactly by: 66
22 can be divided exactly by: 88
End of results


Python 3
Please, enter a number: 12
12 can be divided exactly by: 24
12 can be divided exactly by: 36
12 can be divided exactly by: 48
12 can be divided exactly by: 60
12 can be divided exactly by: 72
12 can be divided exactly by: 84
12 can be divided exactly by: 96
End of results